

AE 623 Project 1

Mesh Generation

Matthew Conley, Eric Heilshorn, Max Mah, Adam Sak, William Zhang

Lab Rats

January 30, 2026

An unstructured triangular mesh was created for a 2-D turbine blade passage. First, an initial "coarse" (1097-cell) mesh was created. Next, an algorithm was implemented to project points from the domain interior to the blade geometry. Local refinement was implemented, improving resolution in important regions. The initial coarse mesh underwent local refinement. This mesh then underwent uniform refinement, yielding 3 finer meshes. Mesh refinement was verified by computing normal vectors and lengths for interior and boundary faces.

1 Introduction

Mesh refinement within CFD simulations is key with regard to accuracy of the output variables. Smaller cells are more accurately able to capture sharp changes in a variable such as U , shear force modeling, and improve near wall resolution. The basis of all CFD simulations is to solve discretized equations, but the method of mesh generation is typically unique between solvers due to different meshing strategies and cell requirements such as non-orthogonality. Meshing strategies need to be able to refine areas of interest depending on the environment simulated. For this problem, refinement needs to be focused on the blade edges, while a more coarse refinement is acceptable further away from the points of interest, such as at the inflow and outflow.

2 Methods

2.1 Initial Mesh Generation

An initial coarse, unstructured, triangular mesh of 1097 cells was created with the following ap-

proach. An uniform cartesian grid of points (using Python's `meshgrid` function) was created to populate the rectangular inflow region, the rectangular outflow region, and the extents of the blade passage region. Using the coordinates provided in `bladeupper.txt` and `bladelower.txt`, the upper and lower surfaces of the blade were splined using `CubicSpline` from `scipy`. Points outside the extents of the upper and lower surfaces of the blade passage were then masked out. The refinement level for the initial placement of nodes was set so that the resulting mesh contained approximately 1000 cells. The node coordinate array V was passed into `scipy`'s `Delaunay` function to create the unstructured triangular mesh and return the "element-to-node" array `E2N`, which stores the three node indices of each element in counterclockwise order. The node-hash connectivity algorithm was employed to create the interior edge array `IE` and the boundary edge array `BE`. It is an $O(\text{nelem})$ algorithm that loops over elements, storing the visited edges by node pairs. A previously-visited edge is identified as an interior edge and an unpaired edge is identified as a boundary. The psuedo-algorithm for the node-hash connectivity function in our code is summarized as follows:

1. **Initialization:** Create a sparse hash matrix H of size $(n_{\text{node}} + 1) \times (n_{\text{node}} + 1)$ to track edge visits. Each entry $H[n_1, n_2]$ stores which element first encountered the edge connecting nodes n_1 and n_2 .
2. **Edge Detection:** Loop over all elements and their three edges. For each edge defined by nodes (n_1, n_2) :
 - If $H[n_1, n_2] = 0$: This is the first visit to the edge. Store the current element

number in both $H[n_1, n_2]$ and $H[n_2, n_1]$. Assume this is a boundary edge initially.

- If $H[n_1, n_2] > 0$: This is the second visit, confirming it is an interior edge shared by two elements. Record $(n_1, n_2, elem_L, elem_R)$ in the IE array, where $elem_L$ is the stored element from H and $elem_R$ is the current element. Set $H[n_1, n_2] = H[n_2, n_1] = -1$ to mark as processed.
- If $H[n_1, n_2] < 0$: Error condition - edge visited more than twice (non-manifold mesh).

3. Boundary Edge Extraction: After processing all elements, scan the upper triangle of H for entries that are still positive (visited once). These are boundary edges. For each such edge, record $(n_1, n_2, elem)$ in the BE array.

After employing a working node-hash connectivity algorithm, the ordering of nodes in the created IE and BE arrays was corrected to follow the same order in which they appear in each column of the E2N array in $[n_1, n_2, n_3]$ in order to keep the counterclockwise convention consistent. With this convention, computation of normal vectors for both interior and boundary edges in subsection 2.2 became straightforward. Periodic boundaries were accounted for as interior edges by modifying the IE and BE arrays; the nodes with matching x-coordinates for both periodic groups were identified in BE, removed, and inserted into IE with neighboring elements assigned according to the matched periodic edge. These data structures mentioned are summarized in Table 1 are consistent with the in-class notes [1].

2.2 Creation of Data Matrices

With E2N created with Delaunay triangulation, and IE and BE created using the node-hash connectivity algorithm, the remaining data structures were created as described below.

1. **Interior Faces to Elements (I2E):** For each interior edge in IE:

Array	Size	Columns
V	[nnode x 2]	[x, y]
E2N	[nelem x 3]	[n1, n2, n3]
IE	[nIE x 4]	[n1, n2, elemL, elemR]
BE	[nBE x 3]	[n1, n2, elem]
I2E	[nIE x 4]	[elemL, faceL, elemR, faceR]
B2E	[nBE x 3]	[elem, face, bgroup]
In	[nIE x 2]	[nx, ny]
Bn	[nBE x 2]	[nx, ny]
Area	[nelem x 1]	[Area]

Table 1: Data structures for mesh pre-processing.

- The element with the smaller index in the two right columns is assigned as **elemL** in I2E
- **elemL** is then used to index the rows of the E2N array to retrieve the three respective node indices
- These nodes were compared with **n1** and **n2** of IE, and the third remaining node was selected, which corresponds to the node opposite to the desired edge.
- Return this node's local index (it's column number, 1-based) in I2E as **faceL**
- Repeat the same procedure for the right element **elemR** to find its local face number **faceR**

Special care was taken to treat edges on periodic boundaries as interior edges. When looping through IE, a periodic edge was identified if the corresponding nodes **n1** and **n2** could not be matched to that element's row in E2N. In this case, the elements' nodes were retrieved from E2N. The node with a dissimilar y coordinate to the other two nodes is the node opposite to the horizontal periodic boundary.

2. **Boundary Faces to Elements (B2E):** For each boundary edge in BE:

- Access corresponding element number in BE

- Use element number to retrieve respective element nodes from E2N
- Determine which column the remaining node is in (not in boundary nodes)
- Assign **bgroup** as either "inflow," "outflow," or "wall" based on x position in the domain

3. **Interior Face Normals (In):** For each interior edge in IE,

- Retrieve edge vertices $\mathbf{p}_1 = (x_1, y_1)$ and $\mathbf{p}_2 = (x_2, y_2)$ using 0-based indexing from **V**.
- Compute edge vector: $\mathbf{e} = \mathbf{p}_2 - \mathbf{p}_1 = (x_2 - x_1, y_2 - y_1) = (\Delta x, \Delta y)$
- Compute perpendicular (unnormalized) by 90 deg **clockwise** rotation: $\mathbf{n} = (\Delta y, -\Delta x)$
- Normalize: $\hat{\mathbf{n}} = \frac{\mathbf{n}}{|\mathbf{n}|}$

4. **Boundary Face Normals (Bn):** For each boundary edge in BE:

- Follow the same procedure as interior edges to compute the normal vector

5. **Element Areas:** For each element in E2N:

- Access each node's corresponding x, y coordinates from **V**
- Generate two vectors $\mathbf{u}$ and $\mathbf{v}$ connected at the tail
- Calculate area as half of the cross-product between the two vectors

With the aforementioned data structures created, functionality to write the **.gri** mesh file format was then implemented. The **.gri** file contains the node coordinates, a list of boundary groups ("inflow", "outflow", or "wall") and faces, and the list of nodes per element (E2N). Periodic pairs are determined via shared x coordinates of periodic boundary nodes. The pairings of periodic boundary edges are stored during their removal from BE.

2.3 Projection and wall distance, sizing function

To find the distance from a given point to a wall, functions are generated based on the point of interest and the splined blade surfaces. These functions are minimized using **scipy.minimize_scalar** and return the x position along the blade that is closest to the point of interest. The length of the projection from the point of interest to the nearest blade surface is as follows:

$$d_{surf} = \sqrt{\min(d_u, d_l)},$$

where d_u is the squared distance to the upper blade surface and d_l is the squared distance to the lower blade surface. This distance to the closest wall therefore allows the local refinement to determine which cells need to be refined. The edge midpoint is used as the point from which the wall distance is measured.

For the sizing function, a secondary refinement near the trailing and leading edges was implemented in order to refine areas of interest.

$$dx = \min(|x - x_{LE}|, |x - x_{TE}|)$$

$$b = 8 |\tanh(dx/4)|$$

The final weighting for refinement is based on a combination of the smoothed and scaled distance to the leading/trailing edge in the x direction and the smallest distance to a blade surface. A final weight of 25% and 75% were selected for the distance to the wall and the leading edge/trailing edge parameter, respectively.

$$d = 0.75 d_{surf} + 0.25 b$$

The sizing then blends the target edge lengths between the minimum and maximum allowable lengths, based on a weighted and shifted hyperbolic tangent function:

$$h(d) = h_{\min} + (h_{\max} - h_{\min}) \tanh\left(\frac{d}{15}\right)$$

In order to achieve refinement just upstream of the leading edge and just downstream of the trailing edge, the x position of the edge midpoint was used as opposed to x_b , the x location of the projected point on the blade.

2.4 Local refinement

During the projection step of the code, a function `flag_cells_for_refinement` compared each edge length in the domain (contained within **IE** and **BE**) to its prescribed maximum acceptable edge length per the sizing function, $h(d)$. The edge was flagged if the length was greater than the prescribed length. From here, the code creates new nodes at the edge midpoints, adding the coordinates of these new nodes to **V**. For new nodes added to wall boundaries, the midpoint of the previous edge is not necessarily on the splined blade surface. In these cases, the new point is projected onto the blade spline. With new nodes created, another round of Delaunay triangulation, edge-hashing (to generate **IE** and **BE**, and data matrix creation is performed. Interior nodes are then smoothed with an under-relaxed translation of each node to the average position of its neighbor nodes. This reduced the number of oblique and sharply acute triangles.

$$\mathbf{V}_i^{\text{new}} = (1 - \omega) \mathbf{V}_i + \omega \left(\frac{1}{N_i} \sum_{j \in \mathcal{N}(i)} \mathbf{V}_j \right)$$

To get the initial 1097 cell mesh to 2016 cells, 7 iterations of local refinement are performed, with each iteration followed by under smoothing with 1000 iterations with $\omega = 0.1$.

2.5 Uniform refinement

With the 2016 cell mesh created with 7 iterations of local refinement and smoothing, uniform mesh refinement was implemented. This was done by adapting the local refinement section of the code; instead of flagging individual edges for refinement based on the sizing function, every edge was flagged. Each triangular element was split into four sub-elements. Nodes that were inserted on bisected boundary edges on the upper and lower surfaces of the blade were projected to the true geometry, ie. the cubic spline of the provided points. The projection functionality described in the above section was recycled for this purpose. Three refinements of the 2k coarse mesh were generated. The 8k mesh, the 32k mesh, and the 128k mesh. Note that smoothing was not run for

uniform refinement, leading to quilt-like patterned regions in the mesh where edges were bisected.

2.6 Mesh Verification

The normal vectors of all edges are accessed in **In** and **Bn** for use in the verification process below:

1. Initialize a zero accumulator **E_e** for each element
2. For each interior edge: add $\mathbf{n} \cdot \mathbf{l}$ to $elem_L$, subtract from $elem_R$
3. For each boundary edge: add outward $\mathbf{n} \cdot \mathbf{l}$ to the adjacent element
4. Verify that $|\mathbf{E}_e| \approx 0$ for all elements (discrete divergence theorem)

These results are shown in section 3.3

3 Results

3.1 Task 1

Figure 1 shows the computational domain for the coarse turbine blade passage, with 1097 elements. The approach described in section 2.1 resulted in an initial mesh that did not cluster elements in any specific areas due to the placement of nodes as a uniform cartesian grid, yielding a rectangular-looking mesh with isosceles triangles embedded in varying orientations. Smaller, sharper triangles are located near the blade walls. Note that the nodes on periodic edges were matched as described in subsection 2.2.

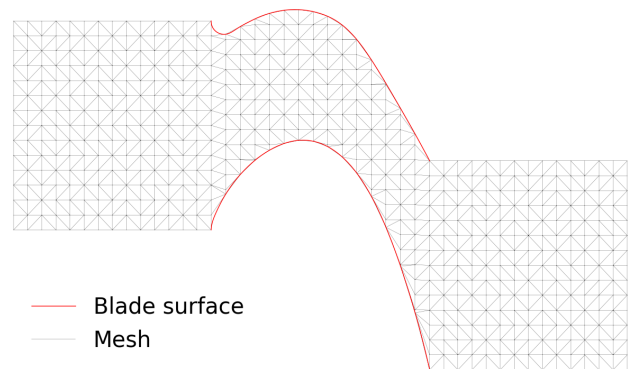


Figure 1: Unstructured coarse mesh

The overarching goal of this coarse mesh has been reached – each element maintains a positive area and proper connectivity, alongside a matching periodic nodeset. This will complete a computationally valid domain that maintains the validation discussed in Task 3.

3.2 Task 2

Table 2 and Table 3 demonstrate the I2E, B2E, areas, and normal vectors required on the `test.gri` mesh provided.

Table 2 demonstrates the B2E connectivity and boundary normal vectors, connecting n_1, n_2 to specific boundary elements. The corresponding normal vectors confirm proper outward orientation—for instance, edges with nodes forming the left boundary exhibit normals pointing in the $-x$ direction, ($n_x = -1.0$), while the top boundaries show upward normals of ($n_y = 1.0$). 45 deg angles are shown as well with diagonal components. The I2E matrix contains a single interior edge (nodes 5-3) shared between elements 2 and 3, and the element area units are all normal.

Table 2: Verification Results for `test.gri` Mesh

B2E Matrix		
n_1	n_2	elem
1	4	1
1	5	1
2	3	2
2	5	2
3	6	3
4	5	1
5	6	3
Boundary Normals		
n_x	n_y	
-1.000	0.000	
-0.707	0.707	
0.000	-1.000	
1.000	0.000	
1.000	0.000	
0.000	1.000	
0.000	1.000	

Table 3: Verification Results Continued

Element Areas			
elem		Area	
1		0.50	
2		0.50	
3		0.50	
I2E Matrix			
n_1	n_2	e_L	e_R
5	3	2	3
Interior Normals			
		n_x	n_y
		0.707	0.707

3.3 Task 3: Coarse Mesh Verification

The resulting verification statistics are shown in Table 4.

Table 4: Mesh Verification Results

Mesh	Max Error	Mean Error
<code>test.gri</code>	0.00×10^0	0.00×10^0
coarse 1k	4.44×10^{-16}	2.12×10^{-18}
2k	2.48×10^{-16}	1.48×10^{-17}
8k	1.24×10^{-16}	7.40×10^{-18}
32k	6.21×10^{-17}	3.46×10^{-18}
128k	3.18×10^{-17}	1.62×10^{-18}

As seen, the cumulative error magnitude is near machine-zero ($\epsilon \approx 10^{-16}$) for the 1k coarse mesh, which is near machine precision. This confirms the mesh topology is consistent, normals are correctly oriented, and the mesh is ready for refinement. If the magnitude error was large, the elements could potentially miscalculate finite-volume fluxes, or missing/intersecting/skewed triangles could be present.

This intuitively aligns with Task 1, whose coarse mesh is generated from the periodic nodes and blade boundary element coordinate positions; the near-zero verification errors confirm that the Delaunay triangulation and subsequent boundary conforming algorithms preserved geometric consistency throughout the mesh generation process. The area formulation discussed in the methodology generates strictly positive (non-negative) areas for triangles, and no degenerate triangles exists. Intuitively, if sum of all edge normal vectors weighted

by edge length around each element should equal zero for a closed polygon, this implies a consistent edge orientation. The cumulative normal vector check in 4 validates the edge orientations are consistent with the mesh topology. This task will be built upon further tasks in refined meshes (2k through 128k cells), which maintain or improve upon this geometric accuracy.

3.4 Task 4: Projection and Wall Distance Calculations

Figure 2 visualizes the $h(d)$ distribution used by the refinement logic. Figure 3 shows the same sizing information applied directly to the mesh edges, where each edge is colored by the target size evaluated at its midpoint.

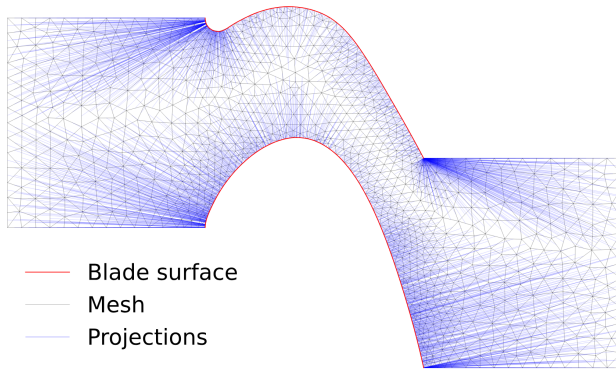


Figure 2: Locally refined mesh ($\approx 2k$ cells) with projections to blade

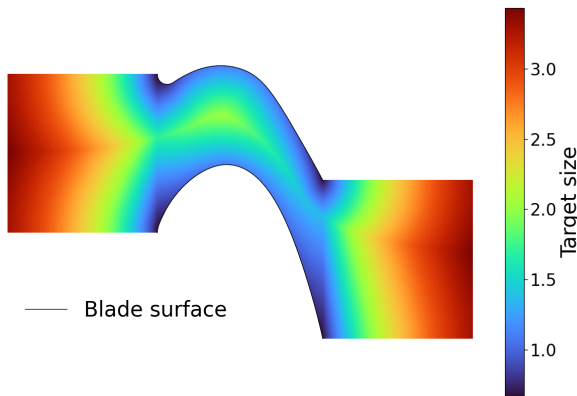


Figure 3: Contour of sizing function $h(d)$

3.5 Task 5: Local Refinement

The figures below show the 2016 cell coarse mesh after local refinement (see subsection 2.4). Note how Figure 1 varies from Figure 4 with refined cells close to the blade wall, especially near the leading and trailing edges.

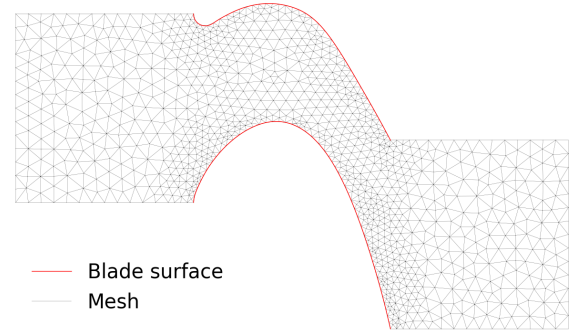


Figure 4: Locally refined mesh ($\approx 2k$ cells)

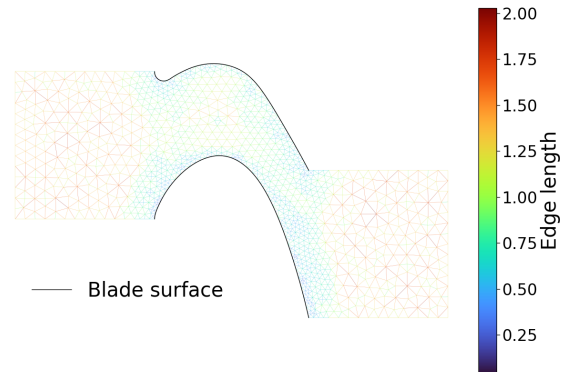


Figure 5: Locally refined mesh ($\approx 2k$ cells), colored by edge length

3.6 Task 6: Uniform Refinements

Starting from the locally refined mesh, three successively finer meshes are generated by applying the same refinement operation uniformly to produce finer triangulations. The uniform refinements for the 2k, 8k, 32k, and 128k meshes all exhibit a maximum and mean error which is near machine precision, as seen in Table 4. The error also decreases by a factor of two for each level of uniform refinement, showing that an increase in resolution leads to improved accuracy.

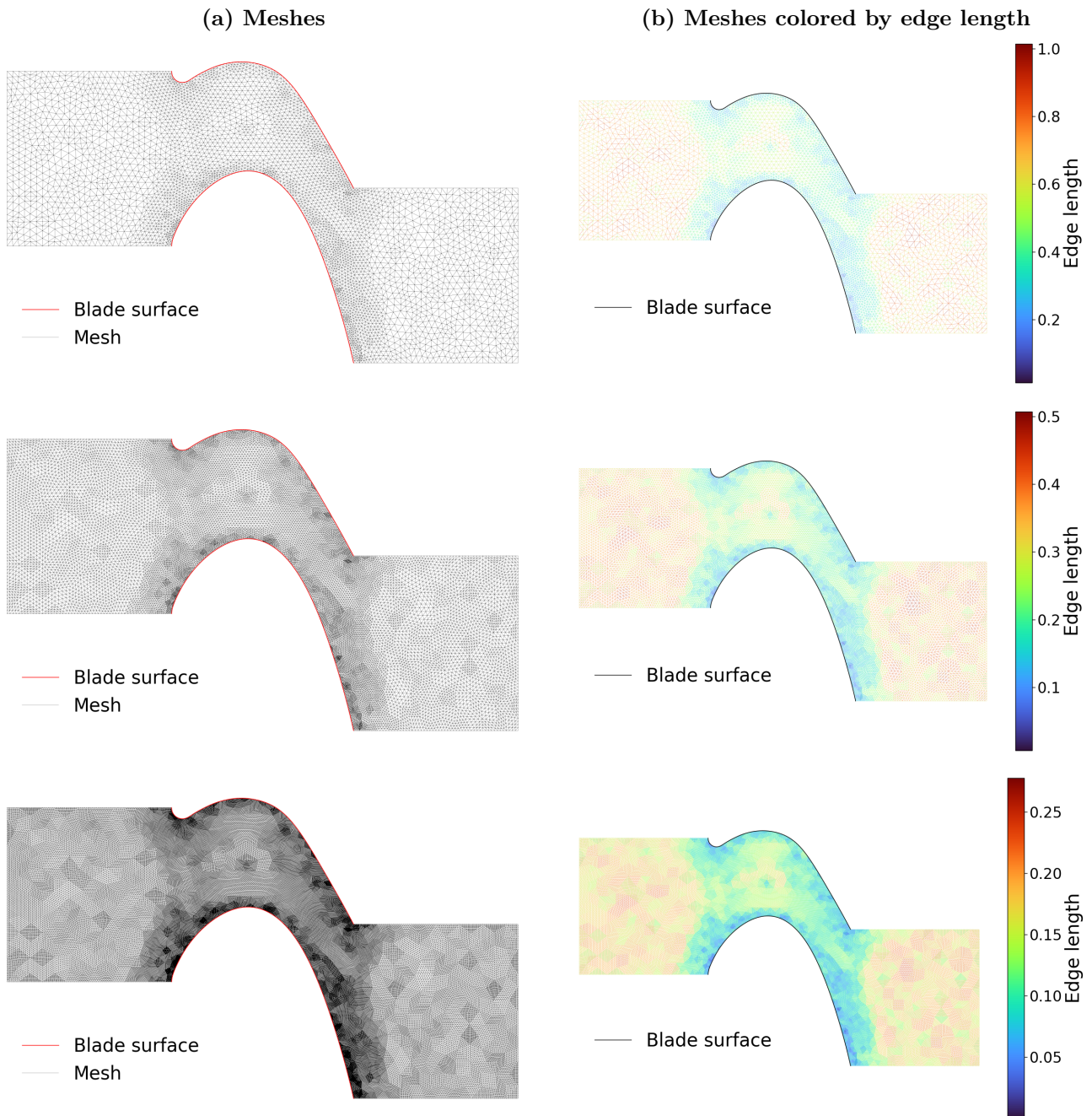


Figure 6: Mesh with various levels of global refinement (8k, 32k, 128k, from top to bottom)

4 Conclusions

A two-dimensional computational domain for a turbine blade passage with periodic boundaries was successfully meshed. An initial coarse, unstructured, triangular mesh was created from the domain extents and was verified via low summed cumulative error. This initial tasking proved critical in eliminating early geometric inconsistencies – if there were defects within the generation of the coarse mesh, the defects would propagate downwards into further refinement stages. This would cause a compromise in the accuracy and stability of the flow solution.

Once the mesh was verified, code was written to generate **I2E**, **B2E**, interior normal **In**, boundary normal **Bn**, and area element matrices. These are then stored in a **.gri format**, which allows a single data file to be written and referenced to in future project iterations; the **I2E** matrix allows flux assembly over interior edges, element areas will be used in flux scaling, and normals will provide relative flux directions per element.

After being stored, wall distances were calculated via projection and combined with x distance relative to leading and trailing edges to specify when local refinement should occur on the mesh. A set of uniform refinements on the locally refined mesh (2016 cells) was used to make 8k, 32k, and 128k element meshes. Each of these global mesh refinement levels took less than 10 seconds in order to generate, while the first local refinement level took roughly 30 seconds.

The coarse or locally refined meshes (1k–2k cells) provide feedback on gross flow features, while the 128k cell mesh provides resolution in secondary flow features. In addition to passing multiple verification tests, these meshing features demonstrate minute errors accumulated throughout the coarse and refined processes. This meshing program and the **.gri** files stored from its completion are now ready to be used with finite volume solvers moving forward.

5 Effort Breakdown

William developed the code for the first three tasks and the verification tests, as well as wrote up the

sections in the report for those tasks. Eric and Max revised the initial meshing methodology, developed the projection method code, and fixed code bugs involving the refinement regions. Matt wrote the code for the local refinement regions. Adam primarily wrote and edited the report.

Table 5: Effort percentages.

	MC	EH	MM	AS	WZ
development	20	20	20	20	20
coding	20	25	25	0	30
managing	10	35	35	10	10
report	10	10	25	35	20

References

- [1] K. Fidkowski, “Advanced computational fluid dynamics,” 2026, aero 623 Course Notes, University of Michigan, Ann Arbor, MI, Jan. 2026.